

ORDINAMENTO

INDICE SEZIONE

- ☐ INTRODUZIONE
- ☐ ORDINAMENTO BASATO SU CONFRONTI
- ☐ MERGE SORT
- ☐ QUICK SORT
- ☐ INSERTION SORT
- ☐ LOWER BOUND PER ALGORITMI BASATI SU CONFRONTI
- ☐ ORDINAMENTO NON BASATO SU CONFRONTI
- ☐ BUCKET SORT
- ☐ RADIX SORT

INTRODUZIONE

L'**ordinamento** è una primitiva fondamentale.

La sua importanza ha motivato un'intensa attività di ricerca per più di mezzo secolo.

In particolare, vedremo:

1. Algoritmi **basati su confronti**:
 - MergeSort
 - QuickSort
 - InsertionSort
 - Lower bound per qualsiasi algoritmo di questo tipo.
2. Algoritmi **non basati su confronti**:
 - BucketSort
 - RadixSort

ORDINAMENTO BASATO SU CONFRONTI

DEFINIZIONE PROBLEMA

Riportiamo la specifica input-output degli algoritmi:

- **INPUT**: Sequenza $S = S[0]S[1] \dots S[n-1]$ di $n \geq 0$ *chiavi* da un *universo ordinato*.
- **OUTPUT**: Sequenza S *ordinata* in senso *non decrescente*.

DEF

Un *algoritmo di ordinamento* basato su *confronti* (**comparison-based**) stabilisce l'ordine delle chiavi di una sequenza S basandosi *esclusivamente* su *confronti tra coppie di chiavi* di S .

Cioè, date *due sequenze* di input S_1 e S_2 , se l'algoritmo esegue gli *stessi confronti*, nello *stesso ordine*, e con lo *stesso esito*, esso determina la *stessa permutazione* che ordina le chiavi.

NOTA: Per comodità, rappresentiamo S come array assumendo di avere accesso diretto a ciascuna chiave $S[i]$. In ogni caso, gli algoritmi affrontati possono essere implementati anche su liste, ma risulterebbero *meno efficienti in pratica*.

DESIGN PATTERN: DIVIDE AND CONQUER

STRATEGIA DIVIDE AND CONQUER

Una strategia di questo tipo, per un problema computazionale Π risolve ogni istanza i di Π di taglia n in *3 fasi*:

1. **DIVIDE:**

- Se $n \leq n_b$ risolve i *direttamente* (CASO BASE).
- Se $n > n_b$, divide i in *2 o più* istanze di Π $i_1, i_2, \dots$ ciascuna di *taglia inferiore* ($< n$).

2. **CONQUER:** Risolve ciascuna istanza i_j con $j \geq 1$.

3. **COMBINE:** Determina la soluzione di i *combinando opportunamente* le soluzioni delle istanze i_j .

OSSERVAZIONE

L'implementazione *più naturale* di tale strategia sfrutta la **ricorsione**. Tuttavia, se la ricorsione porta a risolvere *tante volte la stessa istanza* (come succederebbe per esempio con i numeri di Fibonacci), allora *altre strategie* risultano *più efficienti* (es. *dynamic programming*).

MERGE SORT

Algoritmo: MergeSort(S)

Algorithm MergeSort(S)

```
1: if  $n \leq 1$  then //Divide
2:   return
3: end if
4:  $S_1, S_2 \leftarrow$  sequenze vuote
5: for  $i \leftarrow 0$  to  $\lceil n/2 \rceil - 1$  do
6:    $S_1[i] \leftarrow S[i]$ 
7: end for
8: for  $i \leftarrow \lceil n/2 \rceil$  to  $n - 1$  do
9:    $S_2[i - \lceil n/2 \rceil] \leftarrow S[i]$ 
10: end for
11: MergeSort( $S_1$ ) //Conquer
12: MergeSort( $S_2$ )
13: Merge( $S_1, S_2, S$ ) //Combine
```

La fase **combine** è affidata all'algoritmo Merge (da cui MergeSort prende il nome), che funziona nel seguente modo:

- **INPUT**: Sequenze S_1, S_2 *ordinate* in senso crescente, sequenza vuota S .
- **OUTPUT**: Sequenza $S = S_1 \cup S_2$ *ordinata* in senso crescente.

Algorithm Merge(S_1, S_2, S)

```
1:  $i \leftarrow 0, j \leftarrow 0, k \leftarrow 0$ 
2: while ( $i < S_1.size()$ ) and ( $j < S_2.size()$ ) do
3:   if  $S_1[i] \leq S_2[j]$  then
4:      $S[k++] \leftarrow S_1[i++]$ 
5:   else
6:      $S[k++] \leftarrow S_2[j++]$ 
7:   end if
8: end while
9: while  $i < S_1.size()$  do
10:   $S[k++] \leftarrow S_1[i++]$ 
11: end while
12: while  $j < S_2.size()$  do
13:   $S[k++] \leftarrow S_2[j++]$ 
14: end while
```

COMPLESSITA'

PROPOSIZIONE 1

La *complessità* di Merge(S_1, S_2, S) è $\Theta(n)$, dove n è il *numero di chiavi* in S_1 più il numero di chiavi in S_2 .

✓ DIM.

Ogni *iterazione* dei 3 cicli *while*:

- Esegue $\Theta(1)$ operazioni.
- Sistema *definitivamente* una chiave in S .

Abbiamo allora un *totale* di n *iterazioni* sui 3 cicli, e la complessità è quindi $\Theta(n)$.

□

🔥 PROPOSIZIONE 2

La *complessità* di $\text{MergeSort}(S)$ è $\Theta(n \log n)$, dove $n \geq 1$ è il numero di *chiavi* in S .

✓ DIM.

Consideriamo l'*albero della ricorsione* relativo all'esecuzione di MergeSort per una *sequenza* S *arbitraria* di n chiavi.

Distinguiamo 2 casi:

1. **Caso 1:** $n = 2^d$ per un qualche intero $d \geq 0$.
2. **Caso 2:** $2^d < n < 2^{d+1}$ per un qualche intero $d \geq 0$.

CASO 1: l'albero della ricorsione ha 2^i *nodi* al *livello* i corrispondenti alle invocazioni su istanze di *taglia* $\frac{n}{2^i}$ per $i = 0, 1, \dots, d$: ci sono quindi $d + 1$ *livelli*.

Il *costo associato* a un nodo del livello i è $\Theta\left(\frac{n}{2^i}\right)$, dovuto alle fasi *divide* e *combine*.

Per cui *ogni livello* contribuisce con un *costo aggregato* $\Theta\left(2^i \cdot \frac{n}{2^i}\right) = \Theta(n)$.

La complessità totale è data quindi da:

$$\Theta(d \cdot n)$$

Ma $d \in \Theta(\log n)$.

Allora concludiamo che la *complessità* di MergeSort è:

$$\Theta(n \log n)$$

CASO 2: $2^d < n < 2^{d+1}$.

Assumiamo che ogni livello i dell'albero della ricorsione abbia $\leq 2^i$ *nodi* associati ad istanze di *taglia* $\leq \frac{2^{d+1}}{2^i} < \frac{2n}{2^i}$.

Innanzitutto, siccome $2^d < n$, si ha anche $2^{d+1} < 2n$. Procediamo ora con la dimostrazione del resto della proprietà.

- Per $i = 0$ la proprietà è *verificata banalmente*.
- Per $i = 1$ la sequenza di partenza S è suddivisa in *due sottosequenze* di taglia (rispettivamente) $\lceil n/2 \rceil$ e $\lfloor n/2 \rfloor$, entrambe $< \frac{2n}{2} = n$.

- Assumiamo ora che la proprietà valga per un qualche $1 < i < d + 1$ e mostriamo che allora vale anche per $i + 1$:
 - Ogni nodo ha *al più 2 figli*, per cui se al livello i avevamo $k \leq 2^i$ nodi, ora ne avremo $k' \leq 2k \leq 2^{i+1}$.
 - Poniamo che la *taglia dell'istanza* dei nodi del *livello i* fosse $K \leq \frac{2^{d+1}}{2^i}$. La taglia delle istanze al *livello $i+1$* sarà allora $K' \leq \frac{K}{2} \leq \frac{2^{d+1}}{2^{i+1}}$.

Possiamo allora concludere che il *costo di un livello* è $\Theta\left(2^i \cdot \frac{2^{d+1}}{2^i}\right) = \Theta(2^{d+1})$.

Ma $2^{d+1} < 2n$, per cui il costo è $\Theta(n)$.

Inoltre il caso in cui ci troviamo impone $d = \lfloor \log n \rfloor$.

Allora *anche in questo caso* il *costo complessivo* di MergeSort è:

$$\Theta(n \log n)$$

□

QUICK SORT

Anche questo algoritmo è un esempio di applicazione del paradigma *divide and conquer*:

1. Divide

- $n \leq 1$ è il *caso base*.
- Se $n > 1$:
 - $p \leftarrow S[n - 1]$ (*pivot*)
 - $L \leftarrow \{\text{chiavi} < p \text{ in } S\}$
 - $E \leftarrow \{\text{chiavi} = p \text{ in } S\}$
 - $G \leftarrow \{\text{chiavi} > p \text{ in } S\}$

2. **Conquer**: ordinamento *ricorsivo* delle *sottosequenze* L e G .

3. **Combine**: $S \leftarrow L \circ E \circ G$ (*concatenazione* delle sequenze).

IMPLEMENTAZIONE IN-PLACE

Vediamo come implementare (*in-place*) l'algoritmo $\text{QuickSort}(S, a, b)$.

- **INPUT**: Sequenza S di n chiavi, *indici* $0 \leq a, b < n$
- **OUTPUT**: Sequenza S ordinata in senso non decrescente *tra* $S[a]$ e $S[b]$.

Algorithm QuickSortInPlace(S, a, b)

1: **if** $a \geq b$ **then**

//Divide

2: **return**

3: **end if**

```

4:  $l \leftarrow \text{Partition}(S, a, b)$ 
5:  $\text{QuickSortInPlace}(S, a, l - 1)$  //Conquer
6:  $\text{QuickSortInPlace}(S, l + 1, b)$ 

```

🔗 OSSERVAZIONI

1. Per ordinare *tutta la sequenza* si invoca $\text{QuickSortInPlace}(S, 0, n - 1)$.
2. $\text{Partition}(S, a, b)$ *riorganizza le chiavi* in $S[a \div b]$ intorno ad un *pivot* p , separando quelle $\leq p$ da quelle $\geq p$ e *restituendo l'indice* l del pivot.

Vediamo quindi come funziona $\text{Partition}(S, a, b)$:

- **INPUT:** Sequenza S di n chiavi, indici $0 \leq a, b < n$
- **OUTPUT:** Sequenza S riorganizzata tra a e b , e indice $l \in [a, b]$ tale che $S[i] < S[l] < S[j]$ per ogni i, j con $a \leq i \leq l \leq j \leq b$.

```

Algorithm  $\text{Partition}(S, a, b)$ 
1:  $p \leftarrow S[b]$  //Scelta pivot
2:  $l \leftarrow a, r \leftarrow b - 1$ 
3: while  $l \leq r$  do
4:   while  $l \leq r$  and  $S[l] \leq p$  do
5:      $l \leftarrow l + 1$ 
6:   end while
7:   while  $l \leq r$  and  $S[r] \geq p$  do
8:      $r \leftarrow r - 1$ 
9:   end while
10:  if  $l < r$  then
11:     $\text{Swap}(S[l], S[r])$ 
12:  end if
13: end while
14:  $\text{Swap}(S[l], S[b])$ 
15: return  $l$ 

```

CORRETTEZZA

La correttezza di QuickSort è un'immediata conseguenza della *correttezza di* Partition, la cui dimostrazione si basa sul seguente *invariante*:

🔗 INVARIANTE (PARTITION)

1. $p = S[b]$
2. $S[j] \leq p \forall a \leq j < l$
3. $S[j] \geq p \forall r < j \leq b$

$$4. l \leq r + 1$$

✓ DIM.

E' facile dimostrare che l'invariante è mantenuto alla fine di *ogni iterazione del while esterno*.

Quando questo *termina la sua esecuzione* avremo $l = r + 1$ e la situazione seguente:

$$\underbrace{S[a] \dots S[r]}_{\leq p} \underbrace{S[l] \dots S[b]}_{\geq p} = p$$

Con tutte le *condizioni* dell'invariante *soddisfatte*.

Infine, l'istruzione $\text{Swap}(S[l], S[b])$ renderà l'*output corretto* (indice l).

COMPLESSITA'

🔗 PROPOSIZIONE

La complessità di $\text{Partition}(S, a, b)$ è $\Theta(b - a + 1)$, ovvero *lineare* nel *numero di chiavi* presenti *tra gli indici* a e b .

La proposizione discende dalle seguenti osservazioni:

1. Ad ogni *iterazione del ciclo esterno* l'indice l cresce, e/o r decresce.
2. Il *numero di operazioni* è *proporzionale* al *numero di incrementi/decrementi* dei due indici.
3. Il *numero totale* di incrementi/decrementi è *proporzionale* alla *lunghezza della sottosequenza*.

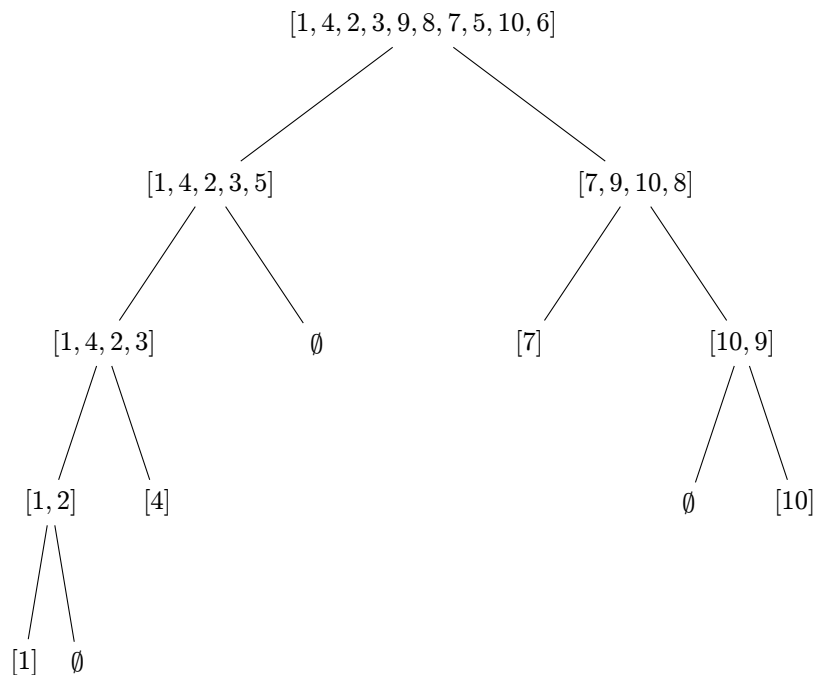
🔗 PROPOSIZIONE

La complessità di $\text{QuickSortInPlace}(S, 0, n - 1)$ è $\Theta(n^2)$.

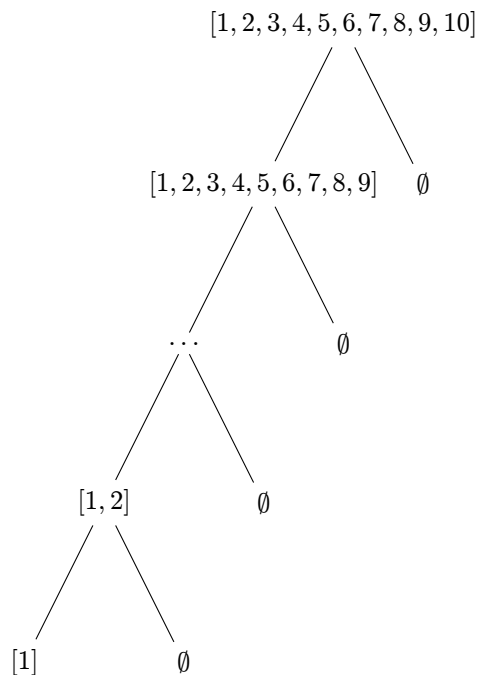
Per l'analisi della complessità dobbiamo, come al solito, considerare l'*albero della ricorsione*.

Facciamo innanzitutto due esempi:

- 1) Albero della ricorsione per $S = [1, 4, 2, 3, 9, 8, 7, 5, 10, 6]$



1) Albero della ricorsione per $S = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$



Procediamo ora con la dimostrazione.

✓ **DIM.**

Consideriamo l'albero della ricorsione per un'istanza arbitraria di taglia n .

- Il *numero di livelli* dell'albero è $\leq n$ perchè da un livello al successivo *almeno 1 pivot* è *sistemato definitivamente*.
- Per lo stesso motivo, la *taglia aggregata* delle *istanze del livello i* è $\leq n - i$.
- Il *costo* contribuito da *un nodo* all'albero della ricorsione è *lineare* nella *taglia dell'istanza* ad esso *associato*, in quanto dominato dalla complessità di Partition.

Allora la complessità è:

$$O\left(\sum_{i=0}^{n-1} (n-i)\right) \in \Theta(n^2)$$

Per dimostrare che è anche $\Omega(n^2)$, consideriamo come *istanza* S la sequenza di n *chiavi ordinate*.

Come si può osservare anche dall'esempio, in questo caso l'albero della ricorsione ha *esattamente* n *livelli*, ed il costo di ciascun livello è $\Theta(n-i)$.

Questa è quindi l'*istanza cattiva* che dimostra che la complessità è $\Omega(n^2)$, e quindi anche $\Theta(n^2)$.

□

OSSERVAZIONE

- L'algoritmo è *in place* *rispetto* alla *memoria heap*, che mantiene le chiavi *sempre nella sequenza* S *di input*.
- Non è tuttavia *in-place* in quanto le variabili a, b, l sono *generate in più copie*, una per *ogni chiamata* ricorsiva, e richiedono uno spazio proporzionale all'altezza dell'albero della ricorsione, che può essere proporzionale a n .

RANDOMIZED QUICKSORT

RandomizedQuicksort è una *variante* di QuickSortInPlace che *evita il caso pessimo probabilisticamente*.

All'interno di Partition, *sostituisce* l'istruzione $p \leftarrow S[b]$ con le seguenti 3:

1. $i \leftarrow$ *intero random* in $[a, b]$
2. $\text{swap}(S[i], S[b])$
3. $p \leftarrow S[b]$

ANALISI

OSSERVAZIONE

Per ogni istanza S esistono *molte esecuzioni possibili*, e quindi molti *tempi diversi* di esecuzione, funzione delle *scelte probabilistiche* (scelta del pivot) fatte dall'algoritmo.

Si tratta quindi di un **algoritmo probabilistico**.

Sia $t_{i,\text{RQS}}$ la *variabile aleatoria* che rappresenta il *numero di operazioni* eseguite da RandomizedQuickSort per risolvere l'istanza i .

PROPOSIZIONE

Per ogni istanza i di taglia n si ha che:

$$\mathbb{E}[t_{i,\text{RQS}}] \in \Theta(n \log n)$$
$$P(t_{i,\text{RQS}} \in O(n \log n)) \geq 1 - \frac{1}{n}$$

La proposizione dice che:

1. Il *valore atteso* di $t_{i,\text{RQS}}$ è $\Theta(n \log n)$
2. La probabilità che $t_{i,\text{RQS}}$ sia $\Theta(n \log n)$ *aumenta* all'*aumentare della taglia* n dell'istanza.

INSERTION SORT

Vediamo l'algoritmo InsertionSort(S).

```
Algorithm InsertionSort( $S$ )
1: for  $i \leftarrow 1$  to  $n - 1$  do
2:   curr  $\leftarrow S[i]$ 
3:    $j \leftarrow i - 1$ 
4:   while ( $j \geq 0$ ) and ( $S[j] > \text{curr}$ ) do
5:      $S[j + 1] \leftarrow S[j]$ 
6:      $j \leftarrow j - 1$ 
7:   end while
8:    $S[j + 1] \leftarrow \text{curr}$ 
9: end for
```

COMPLESSITA'

Nell'*iterazione* i del ciclo *for*, il ciclo *while* esegue *al più i iterazioni*, di costo $\Theta(1)$ ciascuna.

La complessità risulta allora essere:

$$O\left(\sum_{i=1}^{n-1} i\right) = O(n^2)$$

Per l'istanza costituita da chiavi distinte *già ordinate in senso decrescente* si ha che nell'iterazione i del for, il *ciclo while* esegue *esattamente* i iterazioni di costo $\Theta(1)$ ciascuna.

Allora la complessità è anche $\Omega(n^2)$, e quindi $\Theta(n^2)$.

ANALISI PIU' FINE

INVERSIONE

Data una sequenza di n chiavi, una *inversione* (rispetto all'*ordinamento crescente*), è una *coppia di indici* (i, j) con $0 \leq i \neq j < n$ tale che $j < i$ e $S[j] > S[i]$.

Sia K il *numero di inversioni* in una sequenza S di n elementi.

E' immediato vedere che:

$$0 \leq K \leq \binom{n}{2} = \frac{n(n-1)}{2}$$

E si ha:

- S ordinata in *senso crescente* $\implies K = 0$.
- S ordinata in *senso decrescente* $\implies K = \frac{n(n-1)}{2}$.

Proviamo a questo punto ad effettuare un'*analisi della complessità* più precisa, cioè *in funzione* della *taglia* e *anche* del *numero di inversioni* nella sequenza S .

PROPOSIZIONE

La complessità di $\text{InsertionSort}(S)$ è $\Theta(n + K)$, dove n è il numero di chiavi e K è il numero di inversioni in S .

OSSERVAZIONI

- La proposizione *non contraddice* l'analisi fatta sopra, ma *evidenzia* il fatto che InsertionSort è *molto efficiente* quando la sequenza di *input* è *quasi ordinata* (cioè ha poche inversioni).
- L'introduzione di K permette una *partizione più fine delle istanze* e quindi un'analisi più precisa della complessità.

✓ DIM.

Sia S una sequenza di n chiavi con K inversioni.

Le operazioni eseguite da $\text{InsertionSort}(S)$ sono raggruppabili in **2 termini**:

- **A**: operazioni eseguite nei **cicli while**.
- **B**: tutte le operazioni eseguite **fuori** dal **while**.

E' immediato vedere che $B \in \Theta(n)$.

Per ogni $1 \leq i \leq n - 1$ definiamo $K_i :=$ **numero di inversioni** (i, j) con $j < i$ e $S[j] > S[i]$ (numero di inversioni per l'elemento in posizione i).

Il **costo del while** eseguito nell' i -esima **iterazione** del **for** è $\Theta(K_i)$:

$$A \in O\left(\sum_{i=1}^{n-1} K_i\right) = O(K)$$

Dato che ogni inversione è **contata una sola volta** in un K_i .

E concludiamo quindi che la complessità di $\text{InsertionSort}(S)$ è:

$$\Theta(A + B) = \Theta(K + n)$$

□

LOWER BOUND PER ALGORITMI BASATI SU CONFRONTI

🔥 TEOREMA

Per istanze di taglia n , un **qualsiasi** algoritmo A di **ordinamento basato su confronti** richiede

$$\Omega(n \log n)$$

confronti al **caso pessimo**.

Siccome MergeSort, HeapSort, (Randomized) QuickSort e InsertionSort sono **basti su confronti**, dal **teorema sul lower bound** segue che:

- MergeSort, HeapSort, RandomizedQuickSort hanno complessità **asintoticamente ottima**.
- QuickSort (deterministico) e InsertionSort **non** hanno complessità asintoticamente ottima. Tuttavia, per InsertionSort possiamo definire una **famiglia di istanze** abbastanza **ampia** per cui esso risulta **in pratica molto efficiente**.

✓ DIM.

Innanzitutto, *restringiamo il dominio delle istanze*, per ogni taglia n , all'insieme $\mathcal{I}(n)$ delle sequenze S costituite dagli n interi $0, 1, \dots, n-1$, in qualche ordine. Ci sono quindi $n!$ *istanze possibili*.

🔗 OSSERVAZIONE

Se dimostriamo il lower bound per le sequenze di $\mathcal{I}(n)$, esso *varrà anche* per *tutte le possibili istanze*. Infatti ci stiamo semplicemente limitando alle *possibili permutazioni degli elementi* di un certo insieme, senza preoccuparci degli effettivi valori di tali elementi.

Dato che in queste istanze le *chiavi* sono *tutte distinte*, ogni confronto $S[i]$ vs $S[j]$ con $i \neq j$ ha solo *2 esiti possibili*:

- $S[i] > S[j]$
- $S[i] < S[j]$

L'*esecuzione* di A su una qualsiasi sequenza $S \in \mathcal{I}(n)$ può essere *rappresentata* da un **Decision Tree** $T_A(n)$, che è definito come segue:

📖 DECISION TREE

Albero *binario proprio* in cui:

- Ogni *nodo interno* è associato a un *possibile confronto* $S[i]$ vs $S[j]$ eseguito da A . Dopo tale confronto, A procede con l'esecuzione:
 - Sul figlio *sinistro* se $S[i] < S[j]$
 - Sul figlio *destro* se $S[i] > S[j]$
- La *radice* r è associata al *primo confronto* effettuato da A .
- Ogni *foglia* v rappresenta una *terminazione* dell'algoritmo ed è *associata a una permutazione* che serve per *ordinare le chiavi in input*, e che è *determinata univocamente* dai *confronti eseguiti* lungo il *cammino da r a v* e dai loro esiti.

🔗 OSSERVAZIONE CRUCIALE 1

Per ogni sequenza $S \in \mathcal{I}(n)$ esiste un *unico cammino* in $T_A(n)$ dalla *radice* r ad una *foglia* v_s che è associata alla *permutazione che ordina S* .

🔥 OSSERVAZIONE CRUCIALE 2

Date *due sequenze* $S_1, S_2 \in \mathcal{I}(n)$ con $S_1 \neq S_2$ i cammini in $T_A(n)$ associati a esse devono portare per forza a *due foglie* v_{S_1} e v_{S_2} **distinte**.
Infatti, in *caso contrario* si avrebbe una *stessa permutazione* che *ordina entrambe le sequenze*, che è *impossibile*.

Da queste osservazioni ricaviamo immediatamente che $T_A(n)$ deve avere $n!$ *foglie distinte*, una per *ciascuna* $S \in \mathcal{I}(n)$.

Ricordiamo inoltre che in un albero binario proprio con m foglie e altezza h deve valere che $m \leq 2^h$, e allora:

$$h \geq \log_2 m = \log_2 n!$$

Osserviamo inoltre che

$$n! \geq (\lfloor n/2 \rfloor)^{\lfloor n/2 \rfloor}$$

Quindi $T_A(n)$ *deve* avere **altezza**:

$$h \geq \log_2 n! \geq \log_2 (\lfloor n/2 \rfloor)^{\lfloor n/2 \rfloor} \in \Omega(n \log n)$$

E possiamo allora concludere che *deve esistere* una *sequenza* $S \in \mathcal{I}(n)$ per ordinare la quale A esegue $\Omega(n \log n)$ *confronti*.

□

ORDINAMENTO NON BASATO SU CONFRONTI

Assumiamo ora che la sequenza S da ordinare sia costituita da $n \geq 0$ *entry* e che l'ordinamento debba produrre in output la sequenza di entry *ordinate per chiave*.

ORDINAMENTO STABILE

Per lo studio degli algoritmi di questo tipo è utile introdurre la seguente definizione.

📖 ORDINAMENTO STABILE

Un *algoritmo di ordinamento* per sequenze di entry si dice **stabile** se *coppie di entry con la stessa chiave* mantengono in output lo *stesso ordine relativo* che avevano in *input*.

BUCKET SORT

E' un algoritmo che crea N *code* (*bucket*) associate alle chiavi ed esegue *2 scansioni* della sequenza S :

- **1** scansione: *memorizza* le entry nei bucket in base al valore della chiave.
- **2** scansione: *estrae* le entry dai bucket ordinate.

Algoritmo BucketSort(S):

- **INPUT**: sequenza S di n entry con *chiavi intere* in $[0, N - 1]$.
- **OUTPUT**: sequenza S con le entry *ordinate per chiave crescente*.

```
Algorithm BucketSort( $S$ )
1: crea un array  $B$  di  $N$  code  $B[0], \dots, B[N - 1]$ 
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:    $k \leftarrow S[i].\text{getKey}()$ 
4:    $B[k].\text{enqueue}(S[i])$ 
5: end for
6:  $i \leftarrow 0$ 
7: for  $k \leftarrow 0$  to  $N - 1$  do
8:   while  $!(B[k].\text{isEmpty}())$  do
9:      $S[i++] \leftarrow B[k].\text{dequeue}()$ 
10:  end while
11: end for
```

La dimostrazione della *correttezza* è *banale*.

OSSERVAZIONE

I *bucket* sono implementati con *politica FIFO*: questa assunzione ci servirà più avanti.

COMPLESSITA'

PROPOSIZIONE

La *complessità* di BucketSort(S) è $\Theta(n + N)$ dove n è il numero di entry in S e $[0, N - 1]$ è il *range delle chiavi*.

DIM.

Primo ciclo for:

Esegue $\Theta(n)$ operazioni, assumendo che l'inserimento di una entry in un bucket richieda $\Theta(1)$ operazioni.

Secondo ciclo for:

Sia n_k il numero di entry in $B[k]$. Si ha quindi

$$\sum_{k=0}^{N-1} n_k = n$$

L'iterazione k del ciclo costa $\Theta(n_k + 1)$.

Per cui il *secondo ciclo* richiede in *totale*:

$$\Theta\left(\sum_{k=0}^{N-1} (n_k + 1)\right) = \Theta\left(\left(\sum_{k=0}^{N-1} n_k\right) + N\right) = \Theta(n + N)$$

Per cui la *complessità totale* è:

$$\Theta(n + n + N) = \Theta(n + N)$$

□

🔗 OSSERVAZIONI

- L'algoritmo è *efficiente* quando N è *non troppo grande*.
- Se $N = o(n \log n)$ allora BucketSort risulta *asintoticamente migliore di qualsiasi* algoritmo *basato su confronti*.
- BucketSort è *stabile* (conseguenza della politica FIFO).

RADIX SORT

Si tratta di una *generalizzazione* di BucketSort, largamente usato in applicazioni scientifiche.

In questo caso, l'*input* è costituito da n *entry* con *chiavi* costituite da d *cifre* in $[0, N - 1]$ ($c_{d-1}, c_{d-2}, \dots, c_1, c_0$) da *ordinare lessicograficamente*.

L'idea è:

- Dividere l'algoritmo in d *fasi*: ogni fase è *input della successiva*.
- Ogni *fase* è data dall'*ordinamento stabile* su una *cifra diversa*.
- L'*ordine* in cui vengono *scelte le cifre* è *importante*.

Un punto cruciale è dato quindi dalla *scelta dell'ordine*.

Consideriamo il seguente esempio:

$$S = (29, A)(72, F)(63, H)(27, K)$$

In cui abbiamo:

- $n = 4$ (numero entry).
- $d = 2$ (cifre delle chiavi).
- $N = 10$ (range delle cifre).

Approccio 1: partiamo dalla *cifra più significativa* (c_1).

- **I** Fase: ordinamento stabile rispetto a c_1 : $\implies (29, A)(27, K)(63, H)(72, F)$
- **II** Fase: ordinamento stabile rispetto a c_0 : $\implies (72, F)(63, H)(27, K)(29, A)$

Approccio 2: partiamo dalla *cifra meno significativa* (c_0).

- **I** Fase: ordinamento stabile rispetto a c_0 : $\implies (72, F)(63, H)(27, K)(29, A)$
- **II** Fase: ordinamento stabile rispetto a c_1 : $\implies (27, K)(29, A)(63, H)(72, F)$

ALGORITMO

Algoritmo RadixSort(S):

- **INPUT** sequenza S di n entry come detto sopra
- **OUTPUT**: sequenza S con le entry *ordinate lessicograficamente* per chiave.

Algorithm RadixSort(S)

```
1: for  $i \leftarrow 0$  to  $d - 1$  do
2:   ordina  $S$  in modo stabile rispetto a  $c_i$ 
3: end for
```

OSSERVAZIONE

Le *chiavi* possono essere *viste come interi* in $[0, N^d - 1]$ rappresentati in *base N* .

CORRETTEZZA

Prima di dimostrare la correttezza dell'algoritmo, consideriamo una sequenza d'esempio con le seguenti *chiavi*:

$$S_k = 513, 116, 273, 506, 018$$

Abbiamo:

- $n = 5$
- $d = 3$
- $N = 10$

Dopo l' i -esima iterazione del for abbiamo:

ITERAZIONE	SEQUENZA
0	513, 273, 116, 506, 018
1	506, 513, 018, 116, 273
2	018, 116, 273, 506, 513

L'esempio ci aiuta ad intuire il seguente *invariante*:

INVARIANTE

Alla fine dell'*iterazione i -esima*, per $i = 0, 1, \dots, d - 1$, le entry di S sono *ordinate lessicograficamente* rispetto alle cifre $c_i c_{i-1} \dots c_1 c_0$ delle chiavi.

✓ DIM. (Correttezza di RadixSort)

Proviamo la correttezza tramite l'invariante proposto sopra.

- All'*inizio* del for (prima della prima iterazione) l'invariante è *vero per vacuità*.
- Supponiamo ora che *valga alla fine della generica iterazione i* .

Le entry sono allora ordinate rispetto a $c_i c_{i-1} \dots c_1 c_0$.

Alla fine della *successiva iterazione $i + 1$* si ha che:

- 2 entry con **valori diversi** di c_{i+1} saranno ovviamente *ordinate* rispetto a c_{i+1} e, di conseguenza (vista l'ipotesi induttiva), lo saranno *anche* rispetto a $c_{i+1} c_i \dots c_1 c_0$.
- 2 entry con lo **stesso valore** di c_{i+1} risultano *ordinate* grazie all'*invariante* che valeva alla fine dell'iterazione precedente e grazie all'*ordinamento stabile* rispetto a c_{i+1} .

Questo dimostra che l'*invariante* è *vero alla fine di ogni iterazione*.

Per cui è vero anche alla fine dell'*iterazione $d-1$* e la sequenza risulta *correttamente ordinata*.

□

COMPLESSITA'

Supponendo di usare BucketSort per ordinare S in *ciascuna iterazione* del for, la prova della seguente proposizione risulta immediata.

PROPOSIZIONE

La complessità di $\text{RadixSort}(S)$ è $\Theta(d \cdot (n + N))$ dove n è il *numero di entry* in S e le chiavi sono costituite da d *cifre* nel range $[0, N - 1]$.

OSSERVAZIONI

1. Se $d = \Theta(1)$ e $N = O(n)$, allora la complessità è:

$$\Theta(n)$$

2. In generale, se $d = \Theta(1)$ e $N = o(n \log n)$, RadixSort può ordinare le entry con complessità $o(n \log n)$, *battendo qualsiasi algoritmo basato su confronti*.

ESEMPIO

Stimiamo le *complessità relative* di MergeSort e RadixSort per ordinare tutti i *cittadini italiani* usando come chiave il *codice fiscale*.

- Numero cittadini $n \approx 6 \cdot 10^7 \approx 2^{26}$
- Codice fiscale: stringa di *16 caratteri* da un alfabeto di *36* (che associamo agli interni in $[0, 35]$).

MergeSort effettuerà un *numero di operazioni* dell'ordine di:

$$\approx n \log_2(n) \approx 26n$$

Per quanto riguarda RadixSort, vediamo il codice fiscale come composto da d cifre in $[0, N - 1]$. Consideriamo *due casi*.

1 Prendiamo le *cifre singolarmente*: $d = 16$, $N = 36$. La complessità è:

$$\approx 16 \cdot (n + N) \approx 16n$$

2 Raggruppiamo le *cifre a gruppi di 4*: abbiamo allora $d = 4$ cifre in base $N = 36^4$. In tal caso la complessità diventa:

$$\approx d \cdot (n + N) \approx n \cdot \underbrace{\left(d + \frac{Nd}{n}\right)}_{< d+1} < 5 \cdot n$$

In *entrambi i casi vince RadixSort*!